DBMS Mini Project Report

Personal Digital Archive

Team ID: 7

Team Member 1

Name: Bhuvan M S

SRN: PES2UG24CS116

Team Member 2

Name: C K Gagan Gowda

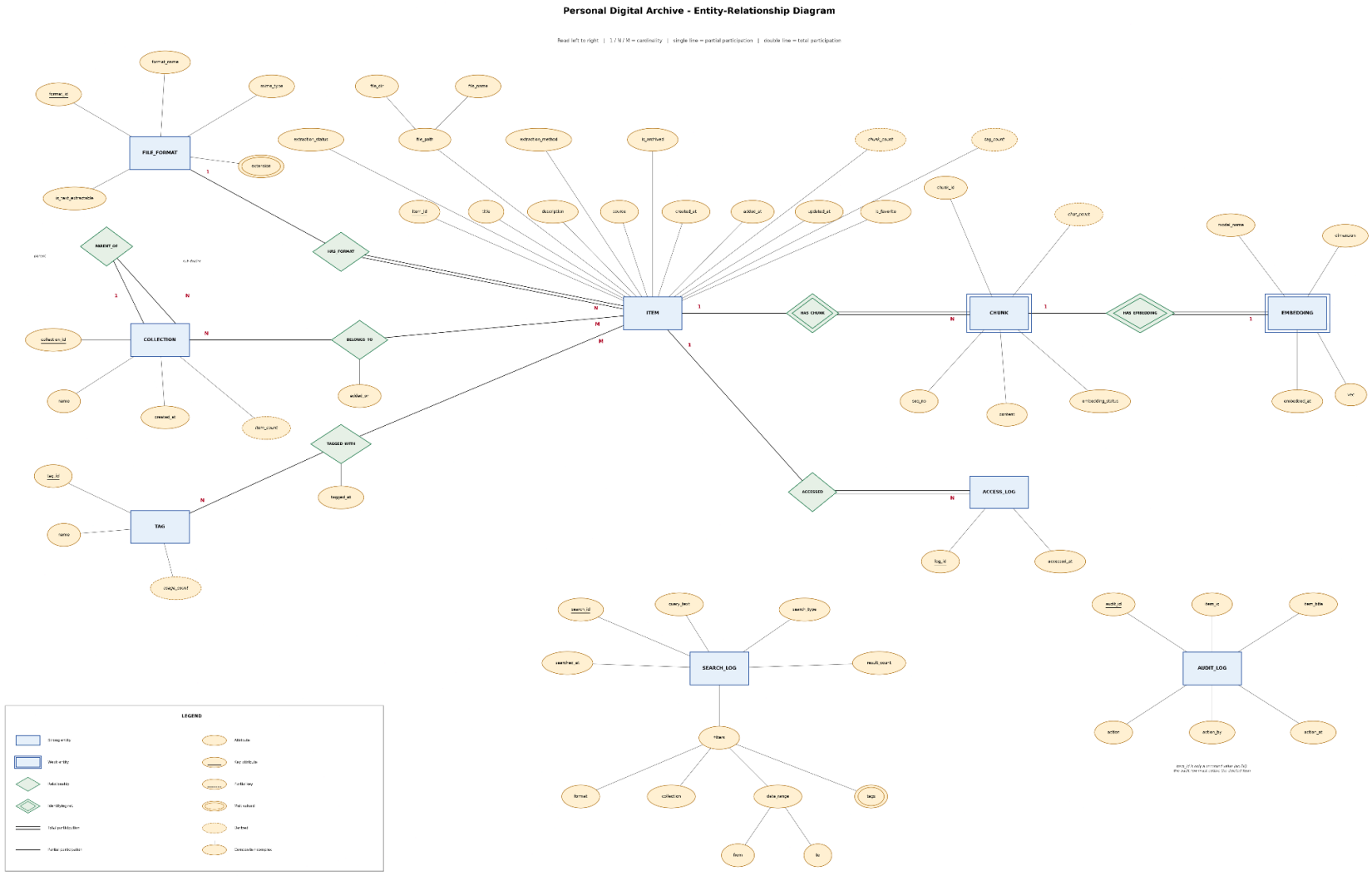
SRN: PES2UG24CS124

1. Problem Statement

Personal files (lecture PDFs, notes, scanned bills, photos, spreadsheets) pile up in folders with arbitrary names, and file-system search cannot answer questions such as "the note where I explained BCNF" or "all receipts tagged last month". The Personal Digital Archive is a local, single-user application that stores the metadata, extracted text (split into chunks), collections, tags and logs of these files in PostgreSQL, and one embedding per chunk in a vector database; the file itself stays on disk. The user can:

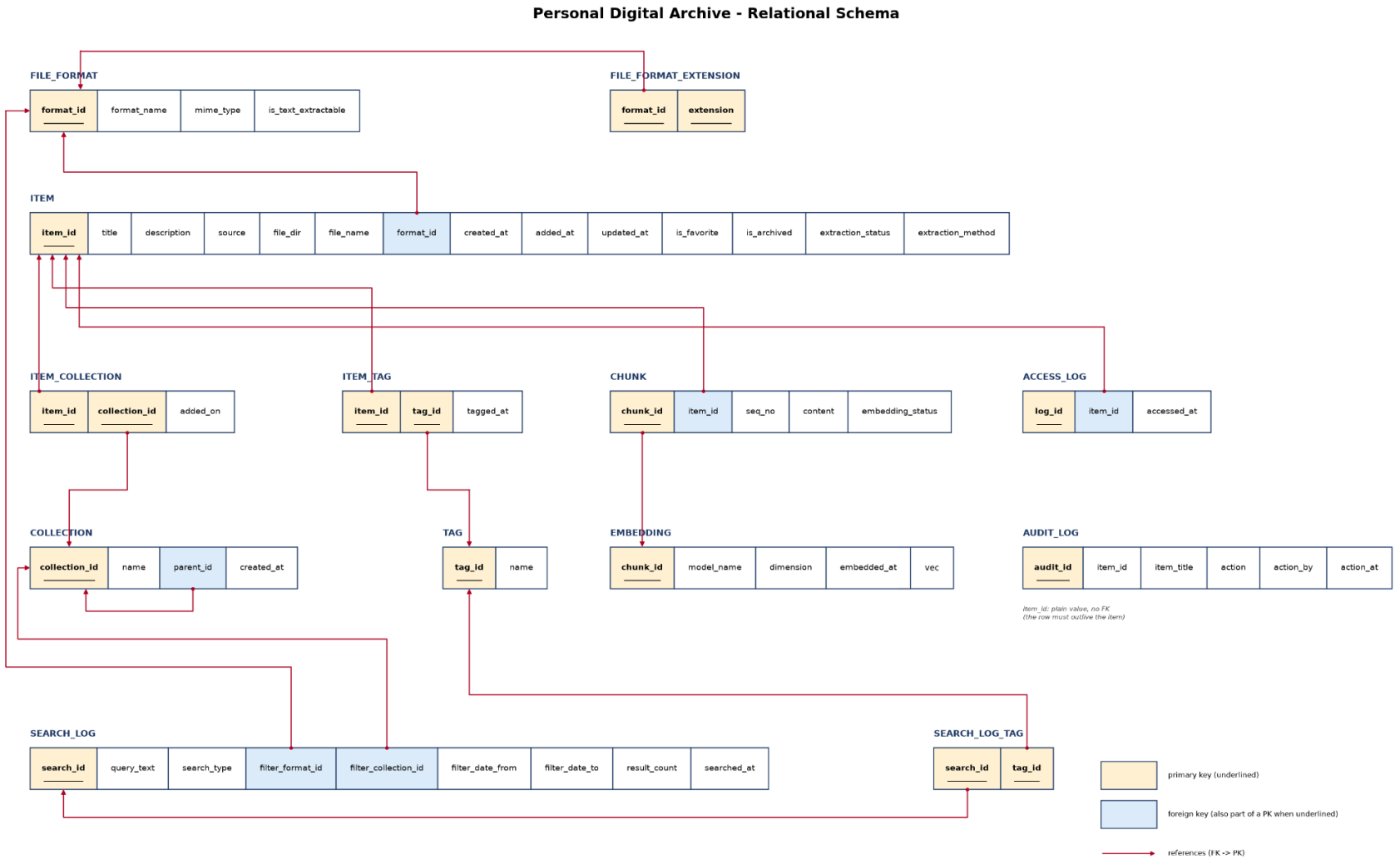
- add, edit, delete and browse items, and mark them favourite or archived;
- organise items in nestable collections and free-form tags (many-to-many);
- search by filters, full-text keywords, semantic similarity, or a hybrid of both;
- keep access and search history and an audit entry for every deleted item, with reports, views, triggers, a stored procedure, transactions and role-based access.

2. ER Diagram



Chen notation, read left to right. CHUNK and EMBEDDING are weak entities; PARENT_OF is recursive; added_on and tagged_at are descriptive attributes; file_path is composite, extension multivalued, filters complex, and the counts derived.

3. Relational Model



Each row of cells is a relation (primary key underlined); arrows run from a foreign key to the key it references. M:N relationships became item_collection and item_tag, the multivalued attribute became file_format_extension, and the weak entities carry their owner key with ON DELETE CASCADE. All relations are in 3NF/BCNF.

4. DDL Commands

```
CREATE DATABASE personal_archive;
\c personal_archive;
```

Output:

```
(Team_7:CS116-CS124)@[local] [mydb] >CREATE DATABASE personal_archive;
CREATE DATABASE
(Team_7:CS116-CS124)@[local] [mydb] >\c personal_archive;
You are now connected to database "personal_archive" as user "myuser".
(Team_7:CS116-CS124)@[local] [personal_archive] >
```

4.1 File format and extensions

```
DROP TABLE IF EXISTS search_log_tag, search_log, audit_log, access_log, embedding, chunk,
                     item_tag, item_collection, item, tag, collection,
                     file_format_extension, file_format CASCADE;

CREATE TABLE file_format (
    format_id         SERIAL         PRIMARY KEY,
    format_name       VARCHAR(50)    NOT NULL,
    mime_type         VARCHAR(100)   NOT NULL UNIQUE,
    is_text_extractable BOOLEAN      NOT NULL DEFAULT TRUE    -- FALSE => metadata only
);

CREATE TABLE file_format_extension (          -- multivalued attribute
    format_id    INT           NOT NULL REFERENCES file_format(format_id) ON DELETE CASCADE,
    extension    VARCHAR(10)   NOT NULL,
    PRIMARY KEY (format_id, extension),
    CONSTRAINT uq_extension UNIQUE (extension)
);
```

Output:

```
(Team_7:CS116-CS124)@[local] [personal_archive] >\d file_format
                                      Table "public.file_format"
      Column       |         Type          | Collation | Nullable |                   Default
-------------------+-----------------------+-----------+----------+-----------------------------------------------
 format_id         | integer               |           | not null | nextval('file_format_format_id_seq'::regclass)
 format_name       | character varying(50) |           | not null |
 mime_type         | character varying(100) |           | not null |
 is_text_extractable | boolean             |           | not null | true
Indexes:
    "file_format_pkey" PRIMARY KEY, btree (format_id)
    "file_format_mime_type_key" UNIQUE CONSTRAINT, btree (mime_type)
Referenced by:
    TABLE "file_format_extension" CONSTRAINT "file_format_extension_format_id_fkey" FOREIGN KEY (format_id) REFERENCES file_format(format_id) ON
 DELETE CASCADE
    TABLE "item" CONSTRAINT "item_format_id_fkey" FOREIGN KEY (format_id) REFERENCES file_format(format_id)
```

```
(Team_7:CS116-CS124)@[local] [personal_archive] >\d file_format_extension
              Table "public.file_format_extension"
  Column   |         Type          | Collation | Nullable | Default
-----------+-----------------------+-----------+----------+---------
 format_id | integer               |           | not null |
 extension | character varying(10) |           | not null |
Indexes:
    "file_format_extension_pkey" PRIMARY KEY, btree (format_id, extension)
    "uq_extension" UNIQUE CONSTRAINT, btree (extension)
Foreign-key constraints:
    "file_format_extension_format_id_fkey" FOREIGN KEY (format_id) REFERENCES file_format(format_id) ON DELETE CASCADE
```

4.2 Collection

```
CREATE TABLE collection (
    collection_id  SERIAL        PRIMARY KEY,
    name           VARCHAR(100)  NOT NULL,
    parent_id      INT           REFERENCES collection(collection_id) ON DELETE CASCADE,
    created_at     TIMESTAMPTZ   NOT NULL DEFAULT now(),
    CONSTRAINT ck_collection_not_own_parent CHECK (parent_id IS NULL OR parent_id <>
collection_id)
);
-- sibling folders must have distinct names (NULL parent = root level)
CREATE UNIQUE INDEX uq_collection_sibling_name
    ON collection (COALESCE(parent_id, 0), lower(name));
```

Output:

```
(Team_7:CS116-CS124)@[local] [personal_archive] >\d collection
                                    Table "public.collection"
    Column     |          Type          | Collation | Nullable |                   Default
---------------+------------------------+-----------+----------+-----------------------------------------------
 collection_id | integer                |           | not null | nextval('collection_collection_id_seq'::regclass)
 name          | character varying(100) |           | not null |
 parent_id     | integer                |           |          |
 created_at    | timestamp with time zone |         | not null | now()
Indexes:
    "collection_pkey" PRIMARY KEY, btree (collection_id)
    "uq_collection_sibling_name" UNIQUE, btree (COALESCE(parent_id, 0), lower(name::text))
Check constraints:
    "ck_collection_not_own_parent" CHECK (parent_id IS NULL OR parent_id <> collection_id)
Foreign-key constraints:
    "collection_parent_id_fkey" FOREIGN KEY (parent_id) REFERENCES collection(collection_id) ON DELETE CASCADE
Referenced by:
    TABLE "collection" CONSTRAINT "collection_parent_id_fkey" FOREIGN KEY (parent_id) REFERENCES collection(collection_id) ON DELETE CASCADE
```

4.3 Tag

```
CREATE TABLE tag (
    tag_id  SERIAL       PRIMARY KEY,
    name    VARCHAR(50)  NOT NULL UNIQUE
);
```

```
(Team_7:CS116-CS124)@[local] [personal_archive] >\d tag
                                  Table "public.tag"
 Column |         Type          | Collation | Nullable |                Default
--------+-----------------------+-----------+----------+---------------------------------------
 tag_id | integer               |           | not null | nextval('tag_tag_id_seq'::regclass)
 name   | character varying(50) |           | not null |
Indexes:
    "tag_pkey" PRIMARY KEY, btree (tag_id)
    "tag_name_key" UNIQUE CONSTRAINT, btree (name)
```

4.4 Item

```
--    file_path is a COMPOSITE attribute (file_dir, file_name)
CREATE TABLE item (
    item_id            SERIAL        PRIMARY KEY,
    title              VARCHAR(255)  NOT NULL,
    description        TEXT,
    source             VARCHAR(255),
    file_dir           VARCHAR(500)  NOT NULL,
    file_name          VARCHAR(255)  NOT NULL,
    format_id          INT           NOT NULL REFERENCES file_format(format_id),
    created_at         TIMESTAMPTZ,                              -- when the file was created
    added_at           TIMESTAMPTZ   NOT NULL DEFAULT now(),     -- when it entered the archive
    updated_at         TIMESTAMPTZ   NOT NULL DEFAULT now(),     -- maintained by trigger
    is_favorite        BOOLEAN       NOT NULL DEFAULT FALSE,
    is_archived        BOOLEAN       NOT NULL DEFAULT FALSE,
    extraction_status  VARCHAR(12)   NOT NULL DEFAULT 'pending',
    extraction_method  VARCHAR(10),
```

```
    search_vector      TSVECTOR GENERATED ALWAYS AS (
                          setweight(to_tsvector('english', coalesce(title,'')), 'A') ||
                          setweight(to_tsvector('english', coalesce(description,'')), 'B')
                       ) STORED,
    CONSTRAINT uq_item_path UNIQUE (file_dir, file_name),
    CONSTRAINT ck_item_extraction_status
        CHECK (extraction_status IN ('pending','done','failed','unsupported')),
    CONSTRAINT ck_item_extraction_method
        CHECK (extraction_method IS NULL OR extraction_method IN ('text_layer','ocr')),
    CONSTRAINT ck_item_dates CHECK (created_at IS NULL OR created_at <= added_at + interval '1
day')
);
```

Output:

```
                                                
```

```
(Team_7:CS116-CS124)@[local] [personal_archive] >\d item_collection
                  Table "public.item_collection"
    Column     |           Type           | Collation | Nullable | Default
---------------+--------------------------+-----------+----------+---------
 item_id       | integer                  |           | not null |
 collection_id | integer                  |           | not null |
 added_on      | timestamp with time zone |           | not null | now()
Indexes:
    "item_collection_pkey" PRIMARY KEY, btree (item_id, collection_id)
Foreign-key constraints:
    "item_collection_collection_id_fkey" FOREIGN KEY (collection_id) REFERENCES collection(collection_id) ON DELETE CASCADE
    "item_collection_item_id_fkey" FOREIGN KEY (item_id) REFERENCES item(item_id) ON DELETE CASCADE
```

```
(Team_7:CS116-CS124)@[local] [personal_archive] >\d item_tag
                      Table "public.item_tag"
  Column   |           Type           | Collation | Nullable | Default
-----------+--------------------------+-----------+----------+---------
 item_id   | integer                  |           | not null |
 tag_id    | integer                  |           | not null |
 tagged_at | timestamp with time zone |           | not null | now()
Indexes:
    "item_tag_pkey" PRIMARY KEY, btree (item_id, tag_id)
Foreign-key constraints:
    "item_tag_item_id_fkey" FOREIGN KEY (item_id) REFERENCES item(item_id) ON DELETE CASCADE
    "item_tag_tag_id_fkey" FOREIGN KEY (tag_id) REFERENCES tag(tag_id) ON DELETE CASCADE
```

4.6 Chunk

```
--    chunk_id is a surrogate candidate key used by the vector DB
CREATE TABLE chunk (
    chunk_id           BIGSERIAL    PRIMARY KEY,
    item_id            INT          NOT NULL REFERENCES item(item_id) ON DELETE CASCADE,
    seq_no             INT          NOT NULL,
    content            TEXT         NOT NULL,
    embedding_status   VARCHAR(10)  NOT NULL DEFAULT 'pending',
    content_tsv        TSVECTOR GENERATED ALWAYS AS (to_tsvector('english', content)) STORED,
    CONSTRAINT uq_chunk_item_seq UNIQUE (item_id, seq_no),
    CONSTRAINT ck_chunk_seq CHECK (seq_no >= 1),
    CONSTRAINT ck_chunk_embedding_status CHECK (embedding_status IN
('pending','done','failed'))
);
```

Output:

```
(Team_7:CS116-CS124)@[local] [personal_archive] >\d chunk
                                                Table "public.chunk"
     Column      |         Type          | Collation | Nullable |                                 Default
-----------------+-----------------------+-----------+----------+-------------------------------------------------------------------------
 chunk_id        | bigint                |           | not null | nextval('chunk_chunk_id_seq'::regclass)
 item_id         | integer               |           | not null |
 seq_no          | integer               |           | not null |
 content         | text                  |           | not null |
 embedding_status | character varying(10) |           | not null | 'pending'::character varying
 content_tsv     | tsvector              |           |          | generated always as (to_tsvector('english'::regconfig, content)) stored
Indexes:
    "chunk_pkey" PRIMARY KEY, btree (chunk_id)
    "uq_chunk_item_seq" UNIQUE CONSTRAINT, btree (item_id, seq_no)
Check constraints:
    "ck_chunk_embedding_status" CHECK (embedding_status::text = ANY (ARRAY['pending'::character varying, 'done'::character varying, 'failed'::ch
aracter varying]::text[]))
    "ck_chunk_seq" CHECK (seq_no >= 1)
Foreign-key constraints:
    "chunk_item_id_fkey" FOREIGN KEY (item_id) REFERENCES item(item_id) ON DELETE CASCADE
```

4.7 Embedding

```
CREATE TABLE embedding (
    chunk_id BIGINT PRIMARY KEY
        REFERENCES chunk(chunk_id)
        ON DELETE CASCADE,
```

```
    model_name VARCHAR(100) NOT NULL,

    dimension INT NOT NULL
        CHECK (dimension > 0),

    embedded_at TIMESTAMPTZ NOT NULL
        DEFAULT now(),

    vec vector(384) NOT NULL,

    CONSTRAINT ck_embedding_dim
        CHECK (vector_dims(vec) = dimension)
);
```

Output:

```
                     Table "public.embedding"
   Column    |           Type           | Collation | Nullable | Default
-------------+--------------------------+-----------+----------+---------
 chunk_id    | bigint                   |           | not null |
 model_name  | character varying(100)   |           | not null |
 dimension   | integer                  |           | not null |
 embedded_at | timestamp with time zone |           | not null | now()
 vec         | vector(384)              |           | not null |
Indexes:
    "embedding_pkey" PRIMARY KEY, btree (chunk_id)
    "idx_embedding_vec" hnsw (vec vector_cosine_ops)
Check constraints:
    "ck_embedding_dim" CHECK (vector_dims(vec) = dimension)
    "embedding_dimension_check" CHECK (dimension > 0)
Foreign-key constraints:
    "embedding_chunk_id_fkey" FOREIGN KEY (chunk_id) REFERENCES chunk(chunk_id) ON DELETE CASCADE
```

4.8 Logs

```
CREATE TABLE access_log (
    log_id       BIGSERIAL    PRIMARY KEY,
    item_id      INT          NOT NULL REFERENCES item(item_id) ON DELETE CASCADE,
    accessed_at  TIMESTAMPTZ  NOT NULL DEFAULT now()
);

-- SEARCH_LOG.filters is a COMPLEX attribute:
--   filters( format, collection, date_range(from,to), {tags} )
CREATE TABLE search_log (
    search_id             BIGSERIAL    PRIMARY KEY,
    query_text            TEXT,
    search_type           VARCHAR(10)  NOT NULL,
    filter_format_id      INT          REFERENCES file_format(format_id)       ON DELETE SET
NULL,
    filter_collection_id  INT          REFERENCES collection(collection_id)    ON DELETE SET
NULL,
    filter_date_from      DATE,
    filter_date_to        DATE,
    result_count          INT          CHECK (result_count >= 0),
    searched_at           TIMESTAMPTZ  NOT NULL DEFAULT now(),
    CONSTRAINT ck_search_type CHECK (search_type IN ('filter','keyword','semantic','hybrid')),
    CONSTRAINT ck_search_dates CHECK (filter_date_from IS NULL OR filter_date_to IS NULL
                                      OR filter_date_from <= filter_date_to)
);

CREATE TABLE search_log_tag (                -- multivalued part of the complex attribute
    search_id  BIGINT  NOT NULL REFERENCES search_log(search_id) ON DELETE CASCADE,
    tag_id     INT     NOT NULL REFERENCES tag(tag_id)           ON DELETE CASCADE,
    PRIMARY KEY (search_id, tag_id)
);
```

```
-- item_id is deliberately NOT a foreign key: the row must outlive the deleted item
CREATE TABLE audit_log (
    audit_id    BIGSERIAL     PRIMARY KEY,
    item_id     INT           NOT NULL,
    item_title  VARCHAR(255),
    action      VARCHAR(10)   NOT NULL,
    action_by   NAME          NOT NULL DEFAULT current_user,
    action_at   TIMESTAMPTZ   NOT NULL DEFAULT now(),
    CONSTRAINT ck_audit_action CHECK (action IN ('DELETE'))
);
```

Output:

```
(Team_7:CS116-CS124)@[local] [personal_archive] >\d access_log
                                    Table "public.access_log"
   Column    |           Type           | Collation | Nullable |                  Default
-------------+--------------------------+-----------+----------+-------------------------------------------
 log_id      | bigint                   |           | not null | nextval('access_log_log_id_seq'::regclass)
 item_id     | integer                  |           | not null |
 accessed_at | timestamp with time zone |           | not null | now()
Indexes:
    "access_log_pkey" PRIMARY KEY, btree (log_id)
Foreign-key constraints:
    "access_log_item_id_fkey" FOREIGN KEY (item_id) REFERENCES item(item_id) ON DELETE CASCADE
```

```
(Team_7:CS116-CS124)@[local] [personal_archive] >\d search_log
                                      Table "public.search_log"
       Column       |           Type           | Collation | Nullable |                  Default
--------------------+--------------------------+-----------+----------+-------------------------------------------
 search_id          | bigint                   |           | not null | nextval('search_log_search_id_seq'::regclass)
 query_text         | text                     |           |          |
 search_type        | character varying(10)    |           | not null |
 filter_format_id   | integer                  |           |          |
 filter_collection_id | integer                |           |          |
 filter_date_from   | date                     |           |          |
 filter_date_to     | date                     |           |          |
 result_count       | integer                  |           |          |
 searched_at        | timestamp with time zone |           | not null | now()
Indexes:
    "search_log_pkey" PRIMARY KEY, btree (search_id)
Check constraints:
    "ck_search_dates" CHECK (filter_date_from IS NULL OR filter_date_to IS NULL OR filter_date_from <= filter_date_to)
    "ck_search_type" CHECK (search_type::text = ANY (ARRAY['filter'::character varying, 'keyword'::character varying, 'semantic'::character vary
ing, 'hybrid'::character varying]::text[]))
    "search_log_result_count_check" CHECK (result_count >= 0)
Foreign-key constraints:
    "search_log_filter_collection_id_fkey" FOREIGN KEY (filter_collection_id) REFERENCES collection(collection_id) ON DELETE SET NULL
    "search_log_filter_format_id_fkey" FOREIGN KEY (filter_format_id) REFERENCES file_format(format_id) ON DELETE SET NULL
```

```
(Team_7:CS116-CS124)@[local] [personal_archive] >\d search_log_tag
          Table "public.search_log_tag"
  Column   |  Type   | Collation | Nullable | Default
-----------+---------+-----------+----------+---------
 search_id | bigint  |           | not null |
 tag_id    | integer |           | not null |
Indexes:
    "search_log_tag_pkey" PRIMARY KEY, btree (search_id, tag_id)
Foreign-key constraints:
    "search_log_tag_search_id_fkey" FOREIGN KEY (search_id) REFERENCES search_log(search_id) ON DELETE CASCADE
    "search_log_tag_tag_id_fkey" FOREIGN KEY (tag_id) REFERENCES tag(tag_id) ON DELETE CASCADE
```

```
(Team_7:CS116-CS124)@[local] [personal_archive] >\d audit_log
                                      Table "public.audit_log"
   Column   |           Type           | Collation | Nullable |                   Default
------------+--------------------------+-----------+----------+-----------------------------------------
 audit_id   | bigint                   |           | not null | nextval('audit_log_audit_id_seq'::regclass)
 item_id    | integer                  |           | not null |
 item_title | character varying(255)   |           |          |
 action     | character varying(10)    |           | not null |
 action_by  | name                     |           | not null | CURRENT_USER
 action_at  | timestamp with time zone |           | not null | now()
Indexes:
    "audit_log_pkey" PRIMARY KEY, btree (audit_id)
Check constraints:
    "ck_audit_action" CHECK (action::text = 'DELETE'::text)
```

4.9 Indexes

```
CREATE INDEX idx_item_format        ON item (format_id);
CREATE INDEX idx_item_added_at      ON item (added_at DESC);
CREATE INDEX idx_item_search_vector ON item USING GIN (search_vector);
CREATE INDEX idx_chunk_content_tsv  ON chunk USING GIN (content_tsv);
CREATE INDEX idx_chunk_pending      ON chunk (chunk_id) WHERE embedding_status = 'pending';
CREATE INDEX idx_collection_parent  ON collection (parent_id);
CREATE INDEX idx_item_tag_tag       ON item_tag (tag_id);
CREATE INDEX idx_item_coll_coll     ON item_collection (collection_id);
CREATE INDEX idx_access_log_item    ON access_log (item_id, accessed_at DESC);
```

4.10 Triggers

```
CREATE OR REPLACE FUNCTION trg_set_updated_at() RETURNS trigger AS $$
BEGIN
    NEW.updated_at := now();
    RETURN NEW;
END $$ LANGUAGE plpgsql;

CREATE TRIGGER item_set_updated_at
    BEFORE UPDATE ON item
    FOR EACH ROW EXECUTE FUNCTION trg_set_updated_at();

CREATE OR REPLACE FUNCTION trg_audit_item_delete() RETURNS trigger AS $$
BEGIN
    INSERT INTO audit_log (item_id, item_title, action)
    VALUES (OLD.item_id, OLD.title, 'DELETE');
    RETURN OLD;
END $$ LANGUAGE plpgsql SECURITY DEFINER;

CREATE TRIGGER item_audit_delete
    AFTER DELETE ON item
    FOR EACH ROW EXECUTE FUNCTION trg_audit_item_delete();
```

4.11 Views

```
CREATE OR REPLACE VIEW item_summary AS
SELECT i.item_id, i.title, ff.format_name, i.extraction_status,
       i.is_favorite, i.is_archived, i.added_at,
       (SELECT count(*) FROM chunk c WHERE c.item_id = i.item_id)           AS chunk_count,
-- derived
       (SELECT count(*) FROM item_tag it WHERE it.item_id = i.item_id)      AS tag_count,
-- derived
       (SELECT count(*) FROM access_log a WHERE a.item_id = i.item_id)      AS access_count
-- derived
FROM item i JOIN file_format ff USING (format_id);

CREATE OR REPLACE VIEW recent_items AS
SELECT i.item_id, i.title, max(a.accessed_at) AS last_accessed
FROM item i JOIN access_log a USING (item_id)
GROUP BY i.item_id, i.title
ORDER BY last_accessed DESC
LIMIT 20;

CREATE OR REPLACE VIEW stats_collection AS
SELECT c.collection_id, c.name, count(ic.item_id) AS item_count          -- derived
FROM collection c LEFT JOIN item_collection ic USING (collection_id)
GROUP BY c.collection_id, c.name;

CREATE OR REPLACE VIEW stats_tag AS
SELECT t.tag_id, t.name, count(it.item_id) AS usage_count                 -- derived
FROM tag t LEFT JOIN item_tag it USING (tag_id)
GROUP BY t.tag_id, t.name;

CREATE OR REPLACE VIEW stats_format AS
SELECT ff.format_id, ff.format_name, count(i.item_id) AS item_count
FROM file_format ff LEFT JOIN item i USING (format_id)
GROUP BY ff.format_id, ff.format_name;
```

4.12 Procedure and function

```
-- Atomic ingest: item + chunks in one transaction. The vector write
-- happens AFTER commit, so chunks start as 'pending'.
CREATE OR REPLACE PROCEDURE add_item(
    p_title        VARCHAR,
    p_description  TEXT,
    p_source       VARCHAR,
    p_file_dir     VARCHAR,
    p_file_name    VARCHAR,
    p_mime_type    VARCHAR,
    p_created_at   TIMESTAMPTZ,
    p_chunks       TEXT[],
    INOUT o_item_id INT DEFAULT NULL
) LANGUAGE plpgsql AS $$
DECLARE
    v_format_id INT;
    v_status    VARCHAR(12);
    v_method    VARCHAR(10);
    v_extract   BOOLEAN;
    i           INT;
BEGIN
    SELECT format_id, is_text_extractable INTO v_format_id, v_extract
    FROM file_format WHERE mime_type = p_mime_type;
    IF NOT FOUND THEN                                    -- unknown type => 'other' (metadata
only)
        SELECT format_id, is_text_extractable INTO v_format_id, v_extract
        FROM file_format WHERE mime_type = 'application/octet-stream';
    END IF;

    IF NOT v_extract THEN
        v_status := 'unsupported'; v_method := NULL;
    ELSIF p_chunks IS NULL OR array_length(p_chunks,1) IS NULL THEN
```

```
        v_status := 'pending';     v_method := NULL;
    ELSE
        v_status := 'done';
        v_method := CASE WHEN p_mime_type LIKE 'image/%' THEN 'ocr' ELSE 'text_layer' END;
    END IF;

    INSERT INTO item (title, description, source, file_dir, file_name,
                      format_id, created_at, extraction_status, extraction_method)
    VALUES (p_title, p_description, p_source, p_file_dir, p_file_name,
            v_format_id, p_created_at, v_status, v_method)
    RETURNING item_id INTO o_item_id;

    IF p_chunks IS NOT NULL THEN
        FOR i IN 1 .. coalesce(array_length(p_chunks,1),0) LOOP
            INSERT INTO chunk (item_id, seq_no, content) VALUES (o_item_id, i, p_chunks[i]);
        END LOOP;
    END IF;
END $$;

-- Hybrid-capable keyword search with optional filters; NULL = "no filter"
CREATE OR REPLACE FUNCTION search_items(
    p_query          TEXT        DEFAULT NULL,
    p_format_id      INT         DEFAULT NULL,
    p_tag_id         INT         DEFAULT NULL,
    p_collection_id  INT         DEFAULT NULL,
    p_date_from      DATE        DEFAULT NULL,
    p_date_to        DATE        DEFAULT NULL
) RETURNS TABLE (item_id INT, title VARCHAR, rank REAL) AS $$
    SELECT i.item_id, i.title,
           CASE WHEN p_query IS NULL THEN 0
                ELSE greatest(
                       ts_rank(i.search_vector, websearch_to_tsquery('english', p_query)),
                       coalesce((SELECT max(ts_rank(c.content_tsv,
websearch_to_tsquery('english', p_query)))
                                 FROM chunk c WHERE c.item_id = i.item_id), 0)
                     )
           END::REAL AS rank
    FROM item i
    WHERE (p_format_id IS NULL OR i.format_id = p_format_id)
      AND (p_tag_id IS NULL OR EXISTS (SELECT 1 FROM item_tag t
                                       WHERE t.item_id = i.item_id AND t.tag_id = p_tag_id))
      AND (p_collection_id IS NULL OR EXISTS (SELECT 1 FROM item_collection ic
                                       WHERE ic.item_id = i.item_id AND ic.collection_id =
p_collection_id))
      AND (p_date_from IS NULL OR i.added_at::date >= p_date_from)
      AND (p_date_to   IS NULL OR i.added_at::date <= p_date_to)
      AND (p_query IS NULL
           OR i.search_vector @@ websearch_to_tsquery('english', p_query)
           OR EXISTS (SELECT 1 FROM chunk c
                      WHERE c.item_id = i.item_id
                        AND c.content_tsv @@ websearch_to_tsquery('english', p_query)))
    ORDER BY 3 DESC, 1;
$$ LANGUAGE sql STABLE;
```

4.13 Roles and privileges

```
DO $$
BEGIN
    IF NOT EXISTS (SELECT 1 FROM pg_roles WHERE rolname='archive_app')       THEN CREATE ROLE
archive_app; END IF;
    IF NOT EXISTS (SELECT 1 FROM pg_roles WHERE rolname='embedding_worker')  THEN CREATE ROLE
embedding_worker; END IF;
    IF NOT EXISTS (SELECT 1 FROM pg_roles WHERE rolname='report_reader')     THEN CREATE ROLE
report_reader; END IF;
    IF NOT EXISTS (SELECT 1 FROM pg_roles WHERE rolname='archive_admin')     THEN CREATE ROLE
archive_admin; END IF;
END $$;
```

```
GRANT USAGE ON SCHEMA public TO archive_app, embedding_worker, report_reader, archive_admin;

-- archive_admin : full control, can read audit_log
GRANT ALL ON ALL TABLES    IN SCHEMA public TO archive_admin;
GRANT ALL ON ALL SEQUENCES IN SCHEMA public TO archive_admin;

-- archive_app : DML on core tables, EXECUTE on add_item / search_items (no audit_log access)
GRANT SELECT, INSERT, UPDATE, DELETE ON
    file_format, file_format_extension, collection, tag, item, item_collection, item_tag,
    chunk, embedding, access_log, search_log, search_log_tag TO archive_app;
GRANT SELECT ON item_summary, recent_items, stats_collection, stats_tag, stats_format TO
archive_app;
GRANT USAGE, SELECT ON ALL SEQUENCES IN SCHEMA public TO archive_app;
GRANT EXECUTE ON PROCEDURE
add_item(VARCHAR,TEXT,VARCHAR,VARCHAR,VARCHAR,VARCHAR,TIMESTAMPTZ,TEXT[],INT) TO archive_app;
GRANT EXECUTE ON FUNCTION  search_items(TEXT,INT,INT,INT,DATE,DATE) TO archive_app;

-- embedding_worker : read chunks, update ONLY embedding_status
GRANT SELECT ON chunk TO embedding_worker;
GRANT UPDATE (embedding_status) ON chunk TO embedding_worker;

-- report_reader : statistics views only
GRANT SELECT ON stats_collection, stats_tag, stats_format TO report_reader;
```

4.14 Verification

```
\dt ->List all tables
\d item ->Display updated structure of item with indexes and triggers
\d chunk -> Display updated structure of chunk with indexes
\dv-> Display all views
\du->Display roles

Output:
List all tables
```

```
                  List of tables
 Schema |          Name           | Type  | Owner
--------+-------------------------+-------+--------
 public | access_log              | table | myuser
 public | audit_log               | table | myuser
 public | chunk                   | table | myuser
 public | collection              | table | myuser
 public | embedding               | table | myuser
 public | file_format             | table | myuser
 public | file_format_extension   | table | myuser
 public | item                    | table | myuser
 public | item_collection         | table | myuser
 public | item_tag                | table | myuser
 public | search_log              | table | myuser
 public | search_log_tag          | table | myuser
 public | tag                     | table | myuser
(13 rows)
```

Display updated structure of item with indexes and triggers

```
                                                
```

```
Indexes:
    "item_pkey" PRIMARY KEY, btree (item_id)
    "idx_item_added_at" btree (added_at DESC)
    "idx_item_format" btree (format_id)
    "idx_item_search_vector" gin (search_vector)
    "uq_item_path" UNIQUE CONSTRAINT, btree (file_dir, file_name)
Check constraints:
    "ck_item_dates" CHECK (created_at IS NULL OR created_at <= (added_at + '1 day'::interval))
    "ck_item_extraction_method" CHECK (extraction_method IS NULL OR (extraction_method::text = ANY (ARRAY['text_layer'::character varying, 'ocr'::cha
racter varying]::text[])))
    "ck_item_extraction_status" CHECK (extraction_status::text = ANY (ARRAY['pending'::character varying, 'done'::character varying, 'failed'::charac
ter varying, 'unsupported'::character varying]::text[]))
Foreign-key constraints:
    "item_format_id_fkey" FOREIGN KEY (format_id) REFERENCES file_format(format_id)
Referenced by:
    TABLE "access_log" CONSTRAINT "access_log_item_id_fkey" FOREIGN KEY (item_id) REFERENCES item(item_id) ON DELETE CASCADE
    TABLE "chunk" CONSTRAINT "chunk_item_id_fkey" FOREIGN KEY (item_id) REFERENCES item(item_id) ON DELETE CASCADE
    TABLE "item_collection" CONSTRAINT "item_collection_item_id_fkey" FOREIGN KEY (item_id) REFERENCES item(item_id) ON DELETE CASCADE
    TABLE "item_tag" CONSTRAINT "item_tag_item_id_fkey" FOREIGN KEY (item_id) REFERENCES item(item_id) ON DELETE CASCADE
Triggers:
    item_audit_delete AFTER DELETE ON item FOR EACH ROW EXECUTE FUNCTION trg_audit_item_delete()
    item_set_updated_at BEFORE UPDATE ON item FOR EACH ROW EXECUTE FUNCTION trg_set_updated_at()
```

Display updated structure of chunk with indexes

```
                                          Table "public.chunk"
      Column      |         Type          | Collation | Nullable |                                Default
------------------+-----------------------+-----------+----------+-----------------------------------------------------------------------
 chunk_id         | bigint                |           | not null | nextval('chunk_chunk_id_seq'::regclass)
 item_id          | integer               |           | not null |
 seq_no           | integer               |           | not null |
 content          | text                  |           | not null |
 embedding_status | character varying(10) |           | not null | 'pending'::character varying
 content_tsv      | tsvector              |           |          | generated always as (to_tsvector('english'::regconfig, content)) stored
Indexes:
    "chunk_pkey" PRIMARY KEY, btree (chunk_id)
    "idx_chunk_content_tsv" gin (content_tsv)
    "idx_chunk_pending" btree (chunk_id) WHERE embedding_status::text = 'pending'::text
    "uq_chunk_item_seq" UNIQUE CONSTRAINT, btree (item_id, seq_no)
Check constraints:
    "ck_chunk_embedding_status" CHECK (embedding_status::text = ANY (ARRAY['pending'::character varying, 'done'::character varying, 'failed'::charact
er varying]::text[]))
    "ck_chunk_seq" CHECK (seq_no >= 1)
Foreign-key constraints:
    "chunk_item_id_fkey" FOREIGN KEY (item_id) REFERENCES item(item_id) ON DELETE CASCADE
Referenced by:
    TABLE "embedding" CONSTRAINT "embedding_chunk_id_fkey" FOREIGN KEY (chunk_id) REFERENCES chunk(chunk_id) ON DELETE CASCADE
```

Display views:

```
personal_archive-# \dv
                List of views
 Schema |        Name        | Type | Owner
--------+--------------------+------+--------
 public | item_summary       | view | myuser
 public | recent_items       | view | myuser
 public | stats_collection   | view | myuser
 public | stats_format       | view | myuser
 public | stats_tag          | view | myuser
(5 rows)
```

Display roles:

```
                                  List of roles
    Role name     |                         Attributes
------------------+------------------------------------------------------------
 archive_admin    | Cannot login
 archive_app      | Cannot login
 embedding_worker | Cannot login
 myuser           | Superuser, Create role, Create DB, Replication, Bypass RLS
 report_reader    | Cannot login
```

5. DML Commands

5.1 INSERT

D1. INSERT - file formats (lookup data)

```
INSERT INTO file_format (format_name, mime_type, is_text_extractable) VALUES
 ('Plain Text', 'text/plain',         TRUE),
 ('Markdown',   'text/markdown',      TRUE),
 ('PDF',        'application/pdf',    TRUE),
 ('Word',       'application/vnd.openxmlformats-officedocument.wordprocessingml.document',
TRUE),
 ('PowerPoint', 'application/vnd.openxmlformats-officedocument.presentationml.presentation',
TRUE),
 ('Excel',      'application/vnd.openxmlformats-officedocument.spreadsheetml.sheet',
TRUE),
 ('CSV',        'text/csv',           TRUE),
 ('HTML',       'text/html',          TRUE),
 ('PNG',        'image/png',          TRUE),
 ('JPEG',       'image/jpeg',         TRUE),
 ('Other',      'application/octet-stream', FALSE);
```

Output:

```
(Team_7:CS116-CS124)@[local] [personal_archive] >SELECT * from file_format;
 format_id | format_name |                               mime_type                                | is_text_extractable
-----------+-------------+------------------------------------------------------------------------+---------------------
         1 | Plain Text  | text/plain                                                             | t
         2 | Markdown    | text/markdown                                                          | t
         3 | PDF         | application/pdf                                                        | t
         4 | Word        | application/vnd.openxmlformats-officedocument.wordprocessingml.document   | t
         5 | PowerPoint  | application/vnd.openxmlformats-officedocument.presentationml.presentation | t
         6 | Excel       | application/vnd.openxmlformats-officedocument.spreadsheetml.sheet          | t
         7 | CSV         | text/csv                                                               | t
         8 | HTML        | text/html                                                              | t
         9 | PNG         | image/png                                                              | t
        10 | JPEG        | image/jpeg                                                             | t
        11 | Other       | application/octet-stream                                               | f
(11 rows)
```

D2. INSERT - multivalued attribute: file extensions

```
INSERT INTO file_format_extension (format_id, extension) VALUES
 (1,'txt'), (2,'md'), (2,'markdown'), (3,'pdf'), (4,'docx'), (5,'pptx'), (6,'xlsx'),
 (7,'csv'), (8,'html'), (8,'htm'), (9,'png'), (10,'jpg'), (10,'jpeg');
```

Output:

```
(Team_7:CS116-CS124)@[local] [personal_archive] >SELECT * FROM file_format_extension;
 format_id | extension
-----------+-----------
         1 | txt
         2 | md
         2 | markdown
         3 | pdf
         4 | docx
         5 | pptx
         6 | xlsx
         7 | csv
         8 | html
         8 | htm
         9 | png
        10 | jpg
        10 | jpeg
(13 rows)
```

D3. INSERT - nested collections (self-reference via parent_id)

```
INSERT INTO collection (name, parent_id) VALUES
 ('Academics',        NULL),   -- 1
 ('DBMS',             1),      -- 2
 ('Machine Learning', 1),      -- 3
 ('Personal',         NULL),   -- 4
 ('Travel',           4),      -- 5
 ('Finance',          4),      -- 6
 ('Goa 2025',         5),      -- 7
 ('Projects',         NULL);   -- 8
```

Output:

```
(Team_7:CS116-CS124)@[local] [personal_archive] >SELECT * FROM collection;
 collection_id |       name       | parent_id |          created_at
---------------+------------------+-----------+-------------------------------
             1 | Academics        |           | 2026-10-04 07:13:29.630168+00
             2 | DBMS             |         1 | 2026-10-04 07:13:29.630168+00
             3 | Machine Learning |         1 | 2026-10-04 07:13:29.630168+00
             4 | Personal         |           | 2026-10-04 07:13:29.630168+00
             5 | Travel           |         4 | 2026-10-04 07:13:29.630168+00
             6 | Finance          |         4 | 2026-10-04 07:13:29.630168+00
             7 | Goa 2025         |         5 | 2026-10-04 07:13:29.630168+00
             8 | Projects         |           | 2026-10-04 07:13:29.630168+00
(8 rows)
```

D4. INSERT - tags

```
INSERT INTO tag (name) VALUES
 ('notes'), ('important'), ('lecture'), ('receipt'), ('travel'), ('research'), ('todo');
```

Output:

```
(Team_7:CS116-CS124)@[local] [personal_archive] >SELECT * FROM tag;
 tag_id |   name
--------+-----------
      1 | notes
      2 | important
      3 | lecture
      4 | receipt
      5 | travel
      6 | research
      7 | todo
(7 rows)
```

D5. Ingest items through the stored procedure add_item (item + chunks in ONE transaction)

```
CALL add_item('DBMS Lecture 1 - ER Model', 'Introduction to entities, relationships and ER
diagrams',
              'College LMS', '/archive/academics/dbms', 'lecture1_er_model.pdf',
'application/pdf',
              '2025-08-12 10:00+05:30',
              ARRAY['An entity is a real-world object that is distinguishable from other
objects.',
                    'A weak entity has no primary key of its own and depends on an owner
entity.',
                    'Relationships associate entities and may carry descriptive attributes.']);

CALL add_item('Normalization Notes', 'Functional dependencies, 3NF and BCNF',
              'Own notes', '/archive/academics/dbms', 'normalization.md', 'text/markdown',
              '2025-09-02 21:15+05:30',
              ARRAY['Normalization removes redundancy by decomposing relations using functional
dependencies.',
                    'BCNF requires every determinant to be a candidate key.']);

CALL add_item('SQL Cheat Sheet', 'Joins, window functions, recursive CTEs',
              'Downloaded', '/archive/academics/dbms', 'sql_cheatsheet.docx',
              'application/vnd.openxmlformats-officedocument.wordprocessingml.document',
              '2025-07-20 09:30+05:30',
              ARRAY['A recursive CTE repeatedly joins its own output to walk hierarchical
data.']);

CALL add_item('Goa Beach Sunset', 'Photo from the Goa trip', 'Phone camera',
'/archive/personal/travel',
              'goa_sunset.jpg', 'image/jpeg', '2025-05-14 18:40+05:30',
              ARRAY['Calangute Beach, Goa - sunset 14 May 2025']);

CALL add_item('Neural Networks Slides', 'Intro deck on neural networks', 'College LMS',
              '/archive/academics/ml', 'nn_intro.pptx',
              'application/vnd.openxmlformats-officedocument.presentationml.presentation',
              '2025-09-10 11:00+05:30',
              ARRAY['A neural network is a stack of layers that learn representations from
data.',
                    'Backpropagation computes gradients of the loss with respect to every
weight.']);

CALL add_item('Family Video', 'Birthday video', 'Phone camera', '/archive/personal/videos',
              'birthday.mkv', 'video/x-matroska', '2025-03-03 19:00+05:30', NULL);

CALL add_item('Old Draft', 'Draft waiting for text extraction', 'Own notes', '/archive/misc',
              'draft_old.txt', 'text/plain', NULL, NULL);
```

Output:

Display Items:

```
(Team_7:CS116-CS124)@[local] [personal_archive] >SELECT i.item_id, i.title, ff.format_name, i.extraction_status, i.extraction_method, i.created_at
FROM item i
JOIN file_format ff USING (format_id)
ORDER BY i.item_id;
 item_id |          title          | format_name | extraction_status | extraction_method |      created_at
---------+-------------------------+-------------+-------------------+-------------------+-----------------------
       1 | DBMS Lecture 1 - ER Model | PDF         | done              | text_layer        | 2025-08-12 04:30:00+00
       2 | Normalization Notes       | Markdown    | done              | text_layer        | 2025-09-02 15:45:00+00
       3 | SQL Cheat Sheet           | Word        | done              | text_layer        | 2025-07-20 04:00:00+00
       4 | Goa Beach Sunset          | JPEG        | done              | ocr               | 2025-05-14 13:10:00+00
       5 | Neural Networks Slides    | PowerPoint  | done              | text_layer        | 2025-09-10 05:30:00+00
       6 | Family Video              | Other       | unsupported       |                   | 2025-03-03 13:30:00+00
       7 | Old Draft                 | Plain Text  | pending           |                   |
(7 rows)
```

Display Chunks:

```
(Team_7:CS116-CS124)@[local] [personal_archive] >SELECT c.chunk_id, c.item_id, c.seq_no, left(c.content, 60) AS content_preview
FROM chunk c
ORDER BY c.item_id, c.seq_no;
 chunk_id | item_id | seq_no |                        content_preview
----------+---------+--------+------------------------------------------------------------
        1 |       1 |      1 | An entity is a real-world object that is distinguishable fro
        2 |       1 |      2 | A weak entity has no primary key of its own and depends on a
        3 |       1 |      3 | Relationships associate entities and may carry descriptive a
        4 |       2 |      1 | Normalization removes redundancy by decomposing relations us
        5 |       2 |      2 | BCNF requires every determinant to be a candidate key.
        6 |       3 |      1 | A recursive CTE repeatedly joins its own output to walk hier
        7 |       4 |      1 | Calangute Beach, Goa - sunset 14 May 2025
        8 |       5 |      1 | A neural network is a stack of layers that learn representat
        9 |       5 |      2 | Backpropagation computes gradients of the loss with respect
(9 rows)
```

D6. INSERT - items added directly (with an explicit extraction method)

```
INSERT INTO item (title, description, source, file_dir, file_name, format_id, created_at,
                  extraction_status, extraction_method)
VALUES
 ('Electricity Bill Sep 2025', 'Scanned bill', 'Scanner', '/archive/personal/finance',
'bill_sep25.pdf',
  3, '2025-09-30 08:00+05:30', 'done', 'ocr'),
 ('Budget 2025', 'Monthly budget spreadsheet', 'Own file', '/archive/personal/finance',
'budget_2025.xlsx',
  6, '2025-01-05 12:00+05:30', 'done', 'text_layer'),
 ('Resume', NULL, 'Own file', '/archive/personal', 'resume.pdf', 3, '2025-06-01 09:00+05:30',
  'done', 'text_layer');
```

D7. INSERT - chunks for the directly inserted items

```
INSERT INTO chunk (item_id, seq_no, content) VALUES
 (8,  1, 'Electricity bill for September 2025. Amount due 1,240 rupees.'),
 (9,  1, 'Budget 2025: rent, groceries, travel, savings and utilities.'),
 (10, 1, 'Resume: Bhuvan M S, B.E. student, interests in databases and machine learning.');
```

Output:

```
 item_id |          title          | extraction_status | extraction_method | seq_no |                                content
---------+-------------------------+-------------------+-------------------+--------+-------------------------------------------------------------
-----------------
       8 | Electricity Bill Sep 2025 | done            | ocr               |      1 | Electricity bill for September 2025. Amount due 1,240 rupees.
       9 | Budget 2025               | done            | text_layer        |      1 | Budget 2025: rent, groceries, travel, savings and utilities.
      10 | Resume                    | done            | text_layer        |      1 | Resume: Bhuvan M S, B.E. student, interests in databases and
machine learning.
(3 rows)
```

D8. INSERT - M:N relationship rows (BELONGS_TO and TAGGED_WITH)

```
INSERT INTO item_collection (item_id, collection_id) VALUES
 (1,2), (2,2), (3,2), (2,3), (5,3), (4,7), (8,6), (9,6), (10,4), (7,8);

INSERT INTO item_tag (item_id, tag_id) VALUES
 (1,1), (1,3), (2,1), (2,3), (2,2), (3,1), (4,5), (5,3), (8,4), (8,2);
```

Output:

```
(Team_7:CS116-CS124)@[local] [personal_archive] >SELECT * FROM item_collectior
 item_id | collection_id |          added_on
---------+---------------+---------------------------
       1 |             2 | 2026-10-04 08:05:44.195+00
       2 |             2 | 2026-10-04 08:05:44.195+00
       3 |             2 | 2026-10-04 08:05:44.195+00
       2 |             3 | 2026-10-04 08:05:44.195+00
       5 |             3 | 2026-10-04 08:05:44.195+00
       4 |             7 | 2026-10-04 08:05:44.195+00
       8 |             6 | 2026-10-04 08:05:44.195+00
       9 |             6 | 2026-10-04 08:05:44.195+00
      10 |             4 | 2026-10-04 08:05:44.195+00
       7 |             8 | 2026-10-04 08:05:44.195+00
(10 rows)
```

```
(Team_7:CS116-CS124)@[local] [personal_archive] >SELECT * FROM item_tag;
 item_id | tag_id |           tagged_at
---------+--------+-------------------------------
       1 |      1 | 2026-10-04 08:05:16.200044+00
       1 |      3 | 2026-10-04 08:05:16.200044+00
       2 |      1 | 2026-10-04 08:05:16.200044+00
       2 |      3 | 2026-10-04 08:05:16.200044+00
       2 |      2 | 2026-10-04 08:05:16.200044+00
       3 |      1 | 2026-10-04 08:05:16.200044+00
       4 |      5 | 2026-10-04 08:05:16.200044+00
       5 |      3 | 2026-10-04 08:05:16.200044+00
       8 |      4 | 2026-10-04 08:05:16.200044+00
       8 |      2 | 2026-10-04 08:05:16.200044+00
(10 rows)
```

D9. INSERT - access log and search log (complex attribute stored across two tables)

```
INSERT INTO access_log (item_id, accessed_at) VALUES
 (1,'2025-10-01 09:00+05:30'), (1,'2025-10-01 20:10+05:30'), (1,'2025-10-02 08:45+05:30'),
 (1,'2025-10-02 21:00+05:30'), (2,'2025-09-30 18:00+05:30'), (2,'2025-10-02 19:30+05:30'),
 (2,'2025-10-03 07:15+05:30'), (8,'2025-10-02 12:00+05:30'), (8,'2025-10-03 09:20+05:30'),
 (4,'2025-09-28 22:00+05:30'), (3,'2025-10-01 16:40+05:30'), (5,'2025-09-29 14:25+05:30');

INSERT INTO search_log (query_text, search_type, filter_format_id, filter_date_from,
filter_date_to, result_count)
VALUES ('normal form', 'keyword', 3, '2025-01-01', '2025-12-31', 2),
       ('how do networks learn', 'semantic', NULL, NULL, NULL, 5);

INSERT INTO search_log_tag (search_id, tag_id) VALUES (1,1), (1,3);
```

Output:
Access_logs:

```
(Team_7:CS116-CS124)@[local] [personal_archive] >SELECT * FROM access_log;
 log_id | item_id |      accessed_at
--------+---------+------------------------
      1 |       1 | 2025-10-01 03:30:00+00
      2 |       1 | 2025-10-01 14:40:00+00
      3 |       1 | 2025-10-02 03:15:00+00
      4 |       1 | 2025-10-02 15:30:00+00
      5 |       2 | 2025-09-30 12:30:00+00
      6 |       2 | 2025-10-02 14:00:00+00
      7 |       2 | 2025-10-03 01:45:00+00
      8 |       8 | 2025-10-02 06:30:00+00
      9 |       8 | 2025-10-03 03:50:00+00
     10 |       4 | 2025-09-28 16:30:00+00
     11 |       3 | 2025-10-01 11:10:00+00
     12 |       5 | 2025-09-29 08:55:00+00
(12 rows)
```

Search_logs

```
 search_id |      query_text       | search_type | filter_format_id | filter_collection_id | filter_date_from | filter_date_to | result_count |
    searched_at
-----------+-----------------------+-------------+------------------+----------------------+------------------+----------------+--------------+------
------------------------
         1 | normal form           | keyword     |                3 |                      | 2025-01-01       | 2025-12-31     |            2 | 2026-
10-04 08:13:01.593536+00
         2 | how do networks learn | semantic    |                  |                      |                  |                |            5 | 2026-
10-04 08:13:01.593536+00
(2 rows)
```

Search log tags

```
 (Team_7:CS116-CS124)@[local] [personal_archive] >SELECT * FROM search_log_tag;
  search_id | tag_id
 -----------+--------
          1 |      1
          1 |      3
 (2 rows)
```

5.2 UPDATE

D10. UPDATE - mark favorite / archived (FR3)

```
UPDATE item SET is_favorite = TRUE WHERE item_id IN (1, 2);
UPDATE item SET is_archived = TRUE WHERE title = 'Family Video';
SELECT item_id, title, is_favorite, is_archived FROM item WHERE is_favorite OR is_archived
ORDER BY item_id;
```

Output:

```
(Team_7:CS116-CS124)@[local] [personal_archive] >SELECT item_id, title, is_favorite, is_archived FROM item WHERE is_favorite OR is_archived ORDER BY
item_id;
 item_id |          title          | is_favorite | is_archived
---------+-------------------------+-------------+-------------
       1 | DBMS Lecture 1 - ER Model | t           | f
       2 | Normalization Notes       | t           | f
       6 | Family Video              | f           | t
(3 rows)
```

D11. UPDATE - edit an item; trigger item_set_updated_at refreshes updated_at (FR19)

```
SELECT item_id, title, updated_at FROM item WHERE item_id = 3;
UPDATE item SET title = 'SQL Cheat Sheet (Revised)', description = 'Joins, windows, CTEs and
triggers' WHERE item_id = 3;
```

```
SELECT item_id, title, updated_at FROM item WHERE item_id = 3;
```

```
(Team_7:CS116-CS124)@[local] [personal_archive] >SELECT item_id, title, updated_at FROM item WHERE item_id = 3;
 item_id |      title      |          updated_at
---------+-----------------+-------------------------------
       3 | SQL Cheat Sheet | 2026-10-04 07:17:03.87712+00
(1 row)

(Team_7:CS116-CS124)@[local] [personal_archive] >UPDATE item SET title = 'SQL Cheat Sheet (Revised)', description = 'Joins, windows, CTEs and trigger
s' WHERE item_id = 3;
UPDATE 1
(Team_7:CS116-CS124)@[local] [personal_archive] >SELECT item_id, title, updated_at FROM item WHERE item_id = 3;
 item_id |           title           |          updated_at
---------+---------------------------+-------------------------------
       3 | SQL Cheat Sheet (Revised) | 2026-10-04 08:24:58.79396+00
(1 row)
```

D12. UPDATE / INSERT - simulate an embedding worker finishing chunks (FR9, FR10)

Simulates adding embeddings of chunks, this will be done by backend of application programmatically by a worker

```
INSERT INTO embedding (
    chunk_id,
    model_name,
    dimension,
    vec
)
SELECT
    c.chunk_id,
    'all-MiniLM-L6-v2',
    384,
    (
        SELECT array_agg(random())::vector(384)
        FROM generate_series(1, 384) g
        WHERE c.chunk_id IS NOT NULL
    )
FROM chunk c
WHERE c.item_id IN (1, 2);

UPDATE chunk
SET embedding_status = 'done'
WHERE item_id IN (1, 2);

UPDATE chunk
SET embedding_status = 'failed'
WHERE item_id = 3;

SELECT
    embedding_status,
    count(*) AS chunks
FROM chunk
GROUP BY embedding_status
ORDER BY 1;
```

Output:

```
(Team_7:CS116-CS124)@[local] [personal_archive] >SELECT embedding_status, count(*) AS chunks FROM chunk GROUP BY embedding_status ORDER BY 1;
 embedding_status | chunks
------------------+--------
 done             |      5
 failed           |      1
 pending          |      6
(3 rows)
```

5.3 SELECT queries

Q1. SELECT - all items with their format (join)

```
SELECT i.item_id, i.title, ff.format_name, i.extraction_status, i.extraction_method
FROM item i JOIN file_format ff USING (format_id)
ORDER BY i.item_id;
```

Output:

```
(Team_7:CS116-CS124)@[local] [personal_archive] >SELECT i.item_id, i.title, ff.format_name, i.extraction_status, i.extraction_method
FROM item i JOIN file_format ff USING (format_id)
ORDER BY i.item_id;
 item_id |          title           | format_name | extraction_status | extraction_method
---------+--------------------------+-------------+-------------------+-------------------
       1 | DBMS Lecture 1 - ER Model | PDF         | done              | text_layer
       2 | Normalization Notes       | Markdown    | done              | text_layer
       3 | SQL Cheat Sheet (Revised) | Word        | done              | text_layer
       4 | Goa Beach Sunset          | JPEG        | done              | ocr
       5 | Neural Networks Slides    | PowerPoint  | done              | text_layer
       6 | Family Video              | Other       | unsupported       |
       7 | Old Draft                 | Plain Text  | pending           |
       8 | Electricity Bill Sep 2025 | PDF         | done              | ocr
       9 | Budget 2025               | Excel       | done              | text_layer
      10 | Resume                    | PDF         | done              | text_layer
(10 rows)
```

Q2. SELECT - filter by format, tag, collection and date range (FR11)

```
SELECT i.item_id, i.title
FROM item i
JOIN item_tag it        ON it.item_id = i.item_id
JOIN tag t              ON t.tag_id = it.tag_id AND t.name = 'notes'
JOIN item_collection ic ON ic.item_id = i.item_id AND ic.collection_id = 2
WHERE i.format_id = 3
  AND i.created_at BETWEEN '2025-08-01' AND '2025-12-31';
```

Output:

```
(Team_7:CS116-CS124)@[local] [personal_archive] >SELECT i.item_id, i.title
FROM item i
JOIN item_tag it        ON it.item_id = i.item_id
JOIN tag t              ON t.tag_id = it.tag_id AND t.name = 'notes'
JOIN item_collection ic ON ic.item_id = i.item_id AND ic.collection_id = 2
WHERE i.format_id = 3
  AND i.created_at BETWEEN '2025-08-01' AND '2025-12-31';
 item_id |          title
---------+--------------------------
       1 | DBMS Lecture 1 - ER Model
(1 row)
```

Q3. Full-text search over title, description and extracted text (FR12)

```
SELECT DISTINCT i.item_id, i.title
FROM item i LEFT JOIN chunk c ON c.item_id = i.item_id
WHERE i.search_vector @@ websearch_to_tsquery('english', 'normalization')
   OR c.content_tsv   @@ websearch_to_tsquery('english', 'normalization')
ORDER BY i.item_id;
```

Output:

```
(Team_7:CS116-CS124)@[local] [personal_archive] >SELECT DISTINCT i.item_id, i.title
FROM item i LEFT JOIN chunk c ON c.item_id = i.item_id
WHERE i.search_vector @@ websearch_to_tsquery('english', 'normalization')
   OR c.content_tsv   @@ websearch_to_tsquery('english', 'normalization')
ORDER BY i.item_id;
 item_id |        title
---------+---------------------
       2 | Normalization Notes
(1 row)
```

Q4. Function search_items(filters) with and without filters (FR20)

```
SELECT * FROM search_items('weak entity');
SELECT * FROM search_items('recursive', p_format_id => 4);
SELECT * FROM search_items(NULL, p_collection_id => 2);
```

Output:

```
(Team_7:CS116-CS124)@[local] [personal_archive] >SELECT * FROM search_items('weak entity');
 item_id |          title           |    rank
---------+--------------------------+------------
       1 | DBMS Lecture 1 - ER Model | 0.10933483
(1 row)

(Team_7:CS116-CS124)@[local] [personal_archive] >SELECT * FROM search_items('recursive', p_format_id => 4);
 item_id |          title           |    rank
---------+--------------------------+------------
       3 | SQL Cheat Sheet (Revised) | 0.06079271
(1 row)

(Team_7:CS116-CS124)@[local] [personal_archive] >SELECT * FROM search_items(NULL, p_collection_id => 2);
 item_id |          title           | rank
---------+--------------------------+------
       1 | DBMS Lecture 1 - ER Model |    0
       2 | Normalization Notes       |    0
       3 | SQL Cheat Sheet (Revised) |    0
(3 rows)
```

Q5. Recursive CTE - list everything under folder 'Academics' and 'Personal' (FR7, FR17)

```
WITH RECURSIVE tree AS (
    SELECT collection_id, name, parent_id, 0 AS depth, name::text AS path
    FROM collection WHERE parent_id IS NULL
  UNION ALL
    SELECT c.collection_id, c.name, c.parent_id, t.depth + 1, t.path || ' / ' || c.name
    FROM collection c JOIN tree t ON c.parent_id = t.collection_id
)
SELECT repeat('    ', depth) || name AS folder, path FROM tree ORDER BY path;
```

Output:

```
(Team_7:CS116-CS124)@[local] [personal_archive] >WITH RECURSIVE tree AS (
    SELECT collection_id, name, parent_id, 0 AS depth, name::text AS path
    FROM collection WHERE parent_id IS NULL
  UNION ALL
    SELECT c.collection_id, c.name, c.parent_id, t.depth + 1, t.path || ' / ' || c.name
    FROM collection c JOIN tree t ON c.parent_id = t.collection_id
)
SELECT repeat('    ', depth) || name AS folder, path FROM tree ORDER BY path;
       folder        |             path
---------------------+---------------------------
 Academics           | Academics
     DBMS            | Academics / DBMS
     Machine Learning | Academics / Machine Learning
 Personal            | Personal
     Finance         | Personal / Finance
     Travel          | Personal / Travel
         Goa 2025    | Personal / Travel / Goa 2025
 Projects            | Projects
(8 rows)
```

Q6. Recursive CTE - all items under 'Personal' including subfolders

```
WITH RECURSIVE sub AS (
    SELECT collection_id FROM collection WHERE name = 'Personal'
  UNION ALL
    SELECT c.collection_id FROM collection c JOIN sub s ON c.parent_id = s.collection_id
)
SELECT DISTINCT i.item_id, i.title
FROM item i JOIN item_collection ic USING (item_id)
WHERE ic.collection_id IN (SELECT collection_id FROM sub)
ORDER BY i.item_id;
```

Output:

```
(Team_7:CS116-CS124)@[local] [personal_archive] >WITH RECURSIVE sub AS (
    SELECT collection_id FROM collection WHERE name = 'Personal'
  UNION ALL
    SELECT c.collection_id FROM collection c JOIN sub s ON c.parent_id = s.collection_id
)
SELECT DISTINCT i.item_id, i.title
FROM item i JOIN item_collection ic USING (item_id)
WHERE ic.collection_id IN (SELECT collection_id FROM sub)
ORDER BY i.item_id;
 item_id |          title
---------+--------------------------
       4 | Goa Beach Sunset
       8 | Electricity Bill Sep 2025
       9 | Budget 2025
      10 | Resume
(4 rows)
```

Q7. Most recently accessed items (aggregate + ORDER BY)

```
SELECT i.item_id, i.title, max(a.accessed_at) AS last_accessed
FROM item i JOIN access_log a USING (item_id)
GROUP BY i.item_id, i.title
ORDER BY last_accessed DESC LIMIT 5;
```

Output:

```
(Team_7:CS116-CS124)@[local] [personal_archive] >SELECT i.item_id, i.title, max(a.accessed_at) AS last_accessed
FROM item i JOIN access_log a USING (item_id)
GROUP BY i.item_id, i.title
ORDER BY last_accessed DESC LIMIT 5;
 item_id |          title          |     last_accessed
---------+-------------------------+-----------------------
       8 | Electricity Bill Sep 2025 | 2025-10-03 03:50:00+00
       2 | Normalization Notes       | 2025-10-03 01:45:00+00
       1 | DBMS Lecture 1 - ER Model | 2025-10-02 15:30:00+00
       3 | SQL Cheat Sheet (Revised) | 2025-10-01 11:10:00+00
       5 | Neural Networks Slides    | 2025-09-29 08:55:00+00
(5 rows)
```

Q8. Top tags by usage with RANK() window function

```
SELECT t.name, count(it.item_id) AS usage_count,
       RANK() OVER (ORDER BY count(it.item_id) DESC) AS usage_rank
FROM tag t LEFT JOIN item_tag it USING (tag_id)
GROUP BY t.name
ORDER BY usage_rank, t.name;
```

Output:

```
(Team_7:CS116-CS124)@[local] [personal_archive] >SELECT t.name, count(it.item_id) AS usage_count,
       RANK() OVER (ORDER BY count(it.item_id) DESC) AS usage_rank
FROM tag t LEFT JOIN item_tag it USING (tag_id)
GROUP BY t.name
ORDER BY usage_rank, t.name;
   name    | usage_count | usage_rank
-----------+-------------+------------
 important |           2 |          1
 lecture   |           2 |          1
 notes     |           2 |          1
 receipt   |           1 |          4
 travel    |           1 |          4
 research  |           0 |          6
(6 rows)
```

Q9. Untagged items (NULL handling with LEFT JOIN)

```
SELECT i.item_id, i.title
FROM item i LEFT JOIN item_tag it USING (item_id)
WHERE it.tag_id IS NULL
ORDER BY i.item_id;
```

Output:

```
(Team_7:CS116-CS124)@[local] [personal_archive] >SELECT i.item_id, i.title
FROM item i LEFT JOIN item_tag it USING (item_id)
WHERE it.tag_id IS NULL
ORDER BY i.item_id;
 item_id |    title
---------+-------------
       6 | Family Video
       7 | Old Draft
       9 | Budget 2025
      10 | Resume
(4 rows)
```

Q10. Items accessed more than average (subquery)

```
SELECT i.item_id, i.title, count(*) AS accesses
FROM item i JOIN access_log a USING (item_id)
GROUP BY i.item_id, i.title
HAVING count(*) > (SELECT avg(cnt) FROM (SELECT count(*) AS cnt FROM access_log GROUP BY
item_id) s)
```

```
ORDER BY accesses DESC;
```

Output:

```
(Team_7:CS116-CS124)@[local] [personal_archive] >SELECT i.item_id, i.title, count(*) AS accesses
FROM item i JOIN access_log a USING (item_id)
GROUP BY i.item_id, i.title
HAVING count(*) > (SELECT avg(cnt) FROM (SELECT count(*) AS cnt FROM access_log GROUP BY item_id) s)
ORDER BY accesses DESC;
 item_id |           title           | accesses
---------+---------------------------+----------
       1 | DBMS Lecture 1 - ER Model |        4
       2 | Normalization Notes       |        3
(2 rows)
```

Q11. Items in collection A (DBMS) but not in B (Machine Learning) - set operation

```
SELECT i.item_id, i.title FROM item i JOIN item_collection ic USING (item_id)
WHERE ic.collection_id = (SELECT collection_id FROM collection WHERE name = 'DBMS')
EXCEPT
SELECT i.item_id, i.title FROM item i JOIN item_collection ic USING (item_id)
WHERE ic.collection_id = (SELECT collection_id FROM collection WHERE name = 'Machine Learning')
ORDER BY item_id;
```

Output:

```
(Team_7:CS116-CS124)@[local] [personal_archive] >SELECT i.item_id, i.title FROM item i JOIN item_collection ic USING (item_id)
WHERE ic.collection_id = (SELECT collection_id FROM collection WHERE name = 'DBMS')
EXCEPT
SELECT i.item_id, i.title FROM item i JOIN item_collection ic USING (item_id)
WHERE ic.collection_id = (SELECT collection_id FROM collection WHERE name = 'Machine Learning')
ORDER BY item_id;
 item_id |           title
---------+---------------------------
       1 | DBMS Lecture 1 - ER Model
       3 | SQL Cheat Sheet (Revised)
(2 rows)
```

Q12. Views (FR18)

```
SELECT * FROM item_summary ORDER BY item_id;
```

Output:

```
 item_id |          title          | format_name | extraction_status | is_favorite | is_archived |          added_at          | chunk_count | ta
g_count | access_count
---------+-------------------------+-------------+-------------------+-------------+-------------+----------------------------+-------------+---
--------+--------------
       1 | DBMS Lecture 1 - ER Model | PDF         | done              | t           | f           | 2026-10-04 07:15:14.985917+00 |           3 |
      2 |            4
       2 | Normalization Notes       | Markdown    | done              | t           | f           | 2026-10-04 07:16:44.270144+00 |           2 |
      3 |            3
       3 | SQL Cheat Sheet (Revised) | Word        | done              | f           | f           | 2026-10-04 07:17:03.87712+00  |           1 |
      1 |            1
       4 | Goa Beach Sunset          | JPEG        | done              | f           | f           | 2026-10-04 07:24:14.233268+00 |           1 |
      1 |            1
       5 | Neural Networks Slides    | PowerPoint  | done              | f           | f           | 2026-10-04 07:24:52.157236+00 |           2 |
      1 |            1
       6 | Family Video              | Other       | unsupported       | f           | t           | 2026-10-04 07:25:06.404159+00 |           0 |
      0 |            0
       7 | Old Draft                 | Plain Text  | pending           | f           | f           | 2026-10-04 07:25:17.26395+00  |           0 |
      0 |            0
       8 | Electricity Bill Sep 2025 | PDF         | done              | f           | f           | 2026-10-04 07:56:19.437795+00 |           1 |
      2 |            2
       9 | Budget 2025               | Excel       | done              | f           | f           | 2026-10-04 07:56:19.437795+00 |           1 |
      0 |            0
      10 | Resume                    | PDF         | done              | f           | f           | 2026-10-04 07:56:19.437795+00 |           1 |
      0 |            0
(10 rows)
```

```
SELECT * FROM recent_items LIMIT 5;
```

Output:

```
(Team_7:CS116-CS124)@[local] [personal_archive] >SELECT * FROM recent_items LIMIT 5;
 item_id |           title           |     last_accessed
---------+---------------------------+-----------------------
       8 | Electricity Bill Sep 2025 | 2025-10-03 03:50:00+00
       2 | Normalization Notes       | 2025-10-03 01:45:00+00
       1 | DBMS Lecture 1 - ER Model | 2025-10-02 15:30:00+00
       3 | SQL Cheat Sheet (Revised) | 2025-10-01 11:10:00+00
       5 | Neural Networks Slides    | 2025-09-29 08:55:00+00
(5 rows)
```

```
SELECT * FROM stats_collection ORDER BY collection_id;
```

Output:

```
(Team_7:CS116-CS124)@[local] [personal_archive] >SELECT * FROM stats_collection ORDER BY collection_id;
 collection_id |       name       | item_count
---------------+------------------+------------
             1 | Academics        |          0
             2 | DBMS             |          3
             3 | Machine Learning |          2
             4 | Personal         |          1
             5 | Travel           |          0
             6 | Finance          |          2
             7 | Goa 2025         |          1
             8 | Projects         |          1
(8 rows)
```

```
SELECT * FROM stats_tag ORDER BY usage_count DESC, name;
```

Output:

```
(Team_7:CS116-CS124)@[local] [personal_archive] >SELECT * FROM stats_tag ORDER BY usage_count DESC, name;
 tag_id |   name    | usage_count
--------+-----------+-------------
      3 | lecture   |           3
      1 | notes     |           3
      2 | important |           2
      4 | receipt   |           1
      5 | travel    |           1
      6 | research  |           0
      7 | todo      |           0
(7 rows)
```

```
SELECT * FROM stats_format WHERE item_count > 0 ORDER BY item_count DESC, format_name;
```

Output:

```
(Team_7:CS116-CS124)@[local] [personal_archive] >SELECT * FROM stats_format WHERE item_count > 0 ORDER BY item_count DESC, format_name;
 format_id | format_name | item_count
-----------+-------------+------------
         3 | PDF         |          3
         6 | Excel       |          1
        10 | JPEG        |          1
         2 | Markdown    |          1
        11 | Other       |          1
         1 | Plain Text  |          1
         5 | PowerPoint  |          1
         4 | Word        |          1
(8 rows)
```

Q13. Hybrid search - SQL filters + ranking by vector similarity (FR14)

```
SELECT
    c.chunk_id,
    c.item_id,
    i.title,
    e.vec <=> (
        SELECT vec
        FROM embedding
        WHERE chunk_id = 1
    ) AS cosine_distance
FROM chunk c
JOIN item i
    USING (item_id)
JOIN item_collection ic
    ON ic.item_id = i.item_id
    AND ic.collection_id = 2
JOIN embedding e
    ON e.chunk_id = c.chunk_id
```

```
WHERE c.embedding_status = 'done'
  AND i.is_archived = FALSE
ORDER BY cosine_distance
LIMIT 5;
```

Output:

```
 chunk_id | item_id |          title           |   cosine_distance
----------+---------+--------------------------+---------------------
        1 |       1 | DBMS Lecture 1 - ER Model |                   0
        3 |       1 | DBMS Lecture 1 - ER Model | 0.22786587580029782
        5 |       2 | Normalization Notes       |  0.2394155821045556
        2 |       1 | DBMS Lecture 1 - ER Model |  0.2544071415800313
        4 |       2 | Normalization Notes       | 0.26734765941079863
(5 rows)
```

5.4 Transactions and concurrency

T1. Transaction - atomic ingest: a failure rolls back the item AND its chunks (FR21)

```
BEGIN;
INSERT INTO item (title, file_dir, file_name, format_id) VALUES ('Half-ingested file',
'/archive/tmp', 'half.txt', 1);
INSERT INTO chunk (item_id, seq_no, content) VALUES (currval('item_item_id_seq'), 0, 'bad
chunk: seq_no 0 violates CHECK');
ROLLBACK;
SELECT count(*) AS leftover_items FROM item WHERE title = 'Half-ingested file';
```

Output:

```
(Team_7:CS116-CS124)@[local] [personal_archive] >BEGIN;
INSERT INTO item (title, file_dir, file_name, format_id) VALUES ('Half-ingested file', '/archive/tmp', 'half.txt', 1);
INSERT INTO chunk (item_id, seq_no, content) VALUES (currval('item_item_id_seq'), 0, 'bad chunk: seq_no 0 violates CHECK');
ROLLBACK;
BEGIN
INSERT 0 1
ERROR:  new row for relation "chunk" violates check constraint "ck_chunk_seq"
DETAIL:  Failing row contains (13, 11, 0, bad chunk: seq_no 0 violates CHECK, pending, '0':5 'bad':1 'check':7 'chunk':2 'seq':3 'violat':6).
ROLLBACK
(Team_7:CS116-CS124)@[local] [personal_archive] >SELECT count(*) AS leftover_items FROM item WHERE title = 'Half-ingested file';
 leftover_items
----------------
              0
(1 row)
```

T2. Concurrency - two workers claim pending chunks with FOR UPDATE SKIP LOCKED (FR22)

```
SELECT chunk_id, item_id, seq_no FROM chunk
WHERE embedding_status = 'pending' ORDER BY chunk_id
LIMIT 2 FOR UPDATE SKIP LOCKED;
```

Output:

```
(Team_7:CS116-CS124)@[local] [personal_archive] >SELECT chunk_id, item_id, seq_no FROM chunk
WHERE embedding_status = 'pending' ORDER BY chunk_id
LIMIT 2 FOR UPDATE SKIP LOCKED;
 chunk_id | item_id | seq_no
----------+---------+--------
        7 |       4 |      1
        8 |       5 |      1
(2 rows)
```

5.5 Query optimisation (EXPLAIN)

X1. EXPLAIN - GIN index on chunk.content_tsv, before and after (NFR)

```
BEGIN;
INSERT INTO chunk (item_id, seq_no, content)
```

```
SELECT 7, g, 'filler paragraph number ' || g || ' about holidays, bills and photographs'
FROM generate_series(1, 20000) g;
UPDATE chunk SET content = 'normalization removes update anomalies' WHERE item_id = 7 AND
seq_no = 777;
ANALYZE chunk;
DROP INDEX idx_chunk_content_tsv;
EXPLAIN ANALYZE SELECT chunk_id FROM chunk WHERE content_tsv @@ to_tsquery('english',
'normalization');
CREATE INDEX idx_chunk_content_tsv ON chunk USING GIN (content_tsv);
ANALYZE chunk;
EXPLAIN ANALYZE SELECT chunk_id FROM chunk WHERE content_tsv @@ to_tsquery('english',
'normalization');
ROLLBACK;
```

Output:

```
(Team_7:CS116-CS124)@[local] [personal_archive] >BEGIN;
INSERT INTO chunk (item_id, seq_no, content)
SELECT 7, g, 'filler paragraph number ' || g || ' about holidays, bills and photographs'
FROM generate_series(1, 20000) g;
UPDATE chunk SET content = 'normalization removes update anomalies' WHERE item_id = 7 AND seq_no = 777;
ANALYZE chunk;
DROP INDEX idx_chunk_content_tsv;
EXPLAIN ANALYZE SELECT chunk_id FROM chunk WHERE content_tsv @@ to_tsquery('english', 'normalization');
CREATE INDEX idx_chunk_content_tsv ON chunk USING GIN (content_tsv);
ANALYZE chunk;
EXPLAIN ANALYZE SELECT chunk_id FROM chunk WHERE content_tsv @@ to_tsquery('english', 'normalization');
ROLLBACK;
```

```
BEGIN
INSERT 0 20000
UPDATE 1
ANALYZE
DROP INDEX
                                                  QUERY PLAN
-------------------------------------------------------------------------------------------------------------
 Seq Scan on chunk  (cost=0.00..831.15 rows=100 width=8) (actual time=0.068..17.732 rows=2.00 loops=1)
   Filter: (content_tsv @@ '''normal'''::tsquery)
   Rows Removed by Filter: 20010
   Buffers: shared hit=581
 Planning:
   Buffers: shared hit=31
 Planning Time: 0.670 ms
 Execution Time: 17.770 ms
(8 rows)
```

```
                                                        QUERY PLAN
-----------------------------------------------------------------------------------------------------------------------------------
 Bitmap Heap Scan on chunk  (cost=13.35..274.98 rows=100 width=8) (actual time=0.020..0.022 rows=2.00 loops=1)
   Recheck Cond: (content_tsv @@ '''normal'''::tsquery)
   Heap Blocks: exact=2
   Buffers: shared hit=5
   ->  Bitmap Index Scan on idx_chunk_content_tsv  (cost=0.00..13.33 rows=100 width=0) (actual time=0.011..0.012 rows=2.00 loops=1)
         Index Cond: (content_tsv @@ '''normal'''::tsquery)
         Index Searches: 1
         Buffers: shared hit=3
 Planning:
   Buffers: shared hit=10
 Planning Time: 0.119 ms
 Execution Time: 0.035 ms
(12 rows)

ROLLBACK
```

5.6 Access control (RBAC)

R1. RBAC - allowed and denied operations (Section 5)

```
SET ROLE archive_app;
SELECT count(*) AS items_visible_to_app FROM item;
SELECT * FROM audit_log;
RESET ROLE;
```

Output:

```
(Team_7:CS116-CS124)@[local] [personal_archive] >SET ROLE archive_app;
SET
(Team_7:CS116-CS124)@[local] [personal_archive] >SELECT count(*) AS items_visible_to_app FROM item;
 items_visible_to_app
----------------------
                   10
(1 row)

(Team_7:CS116-CS124)@[local] [personal_archive] >SELECT * FROM audit_log;
ERROR:  permission denied for table audit_log
(Team_7:CS116-CS124)@[local] [personal_archive] >RESET ROLE;
RESET
```

```
SET ROLE embedding_worker;
UPDATE chunk SET embedding_status = 'done' WHERE chunk_id = 1;
UPDATE chunk SET content = 'tampered' WHERE chunk_id = 1;
DELETE FROM chunk WHERE chunk_id = 1;
RESET ROLE;
```

Output:

```
(Team_7:CS116-CS124)@[local] [personal_archive] >RESET ROLE;
RESET
(Team_7:CS116-CS124)@[local] [personal_archive] >
(Team_7:CS116-CS124)@[local] [personal_archive] >SET ROLE embedding_worker;
SET
(Team_7:CS116-CS124)@[local] [personal_archive] >UPDATE chunk SET embedding_status = 'done' WHERE chunk_id = 1;
UPDATE 1
(Team_7:CS116-CS124)@[local] [personal_archive] >UPDATE chunk SET content = 'tampered' WHERE chunk_id = 1;
ERROR:  permission denied for table chunk
(Team_7:CS116-CS124)@[local] [personal_archive] >DELETE FROM chunk WHERE chunk_id = 1;
ERROR:  permission denied for table chunk
(Team_7:CS116-CS124)@[local] [personal_archive] >RESET ROLE;
RESET
```

```
SET ROLE report_reader;
SELECT * FROM stats_format WHERE item_count > 0 ORDER BY item_count DESC, format_name LIMIT 3;
SELECT * FROM item;
RESET ROLE;

UPDATE chunk SET embedding_status = 'done' WHERE chunk_id = 1;
```

Output:

```
(Team_7:CS116-CS124)@[local] [personal_archive] >SET ROLE report_reader;
SET
(Team_7:CS116-CS124)@[local] [personal_archive] >SELECT * FROM stats_format WHERE item_count > 0 ORDER BY item_count DESC, format_name LIMIT 3;
ERROR:  permission denied for view stats_format
(Team_7:CS116-CS124)@[local] [personal_archive] >SELECT * FROM item;
ERROR:  permission denied for table item
(Team_7:CS116-CS124)@[local] [personal_archive] >RESET ROLE;
RESET
(Team_7:CS116-CS124)@[local] [personal_archive] >UPDATE chunk SET embedding_status = 'done' WHERE chunk_id = 1;
UPDATE 1
```

5.7 DELETE

D13. DELETE - remove an item: cascades to chunks / tags / collections, trigger writes audit_log (FR16)

```
SELECT item_id, title FROM item WHERE item_id = 1;
SELECT count(*) AS chunks_before FROM chunk WHERE item_id = 1;
SELECT count(*) AS embeddings_before FROM embedding e JOIN chunk c USING (chunk_id) WHERE
c.item_id = 1;

DELETE FROM item WHERE item_id = 1;
SELECT count(*) AS chunks_after FROM chunk WHERE item_id = 1;
SELECT count(*) AS embeddings_after FROM embedding e WHERE e.chunk_id NOT IN (SELECT chunk_id
FROM chunk);
SELECT count(*) AS tag_rows_after FROM item_tag WHERE item_id = 1;
SELECT audit_id, item_id, item_title, action, action_by, action_at FROM audit_log;
```

Output:

```
(Team_7:CS116-CS124)@[local] [personal_archive] >SELECT item_id, title FROM item WHERE item_id = 1;
 item_id |           title
---------+---------------------------
       1 | DBMS Lecture 1 - ER Model
(1 row)

(Team_7:CS116-CS124)@[local] [personal_archive] >SELECT count(*) AS chunks_before FROM chunk WHERE item_id = 1;
 chunks_before
---------------
             3
(1 row)

(Team_7:CS116-CS124)@[local] [personal_archive] >SELECT count(*) AS embeddings_before FROM embedding e JOIN chunk c USING (chunk_id) WHERE c.item_id
= 1;
 embeddings_before
-------------------
                 3
(1 row)

(Team_7:CS116-CS124)@[local] [personal_archive] >DELETE FROM item WHERE item_id = 1;
DELETE 1
```

```
(Team_7:CS116-CS124)@[local] [personal_archive] >SELECT count(*) AS chunks_after FROM chunk WHERE item_id = 1;
 chunks_after
--------------
            0
(1 row)

(Team_7:CS116-CS124)@[local] [personal_archive] >SELECT count(*) AS embeddings_after FROM embedding e WHERE e.chunk_id NOT IN (SELECT chunk_id FROM c
hunk);
 embeddings_after
------------------
                0
(1 row)

(Team_7:CS116-CS124)@[local] [personal_archive] >SELECT count(*) AS tag_rows_after FROM item_tag WHERE item_id = 1;
 tag_rows_after
----------------
              0
(1 row)

(Team_7:CS116-CS124)@[local] [personal_archive] >SELECT audit_id, item_id, item_title, action, action_by, action_at FROM audit_log;
(Team_7:CS116-CS124)@[local] [personal_archive] >SELECT audit_id, item_id, item_title, action, action_by, action_at FROM audit_log;
 audit_id | item_id |        item_title         | action | action_by |           action_at
----------+---------+---------------------------+--------+-----------+-------------------------------
        1 |       1 | DBMS Lecture 1 - ER Model | DELETE | myuser    | 2026-10-04 11:07:16.619883+00
(1 row)
```

D14. DELETE - tag and a nested collection (cascades)

```
DELETE FROM tag WHERE name = 'todo';
DELETE FROM collection WHERE name = 'Travel';
SELECT collection_id, name, parent_id FROM collection ORDER BY collection_id;
SELECT item_id, collection_id FROM item_collection WHERE item_id = 4;
```

Output:

```
(Team_7:CS116-CS124)@[local] [personal_archive] >DELETE FROM tag WHERE name = 'todo';
DELETE 1
(Team_7:CS116-CS124)@[local] [personal_archive] >DELETE FROM collection WHERE name = 'Travel';
DELETE 1
(Team_7:CS116-CS124)@[local] [personal_archive] >SELECT collection_id, name, parent_id FROM collection ORDER BY collection_id;
 collection_id |       name       | parent_id
---------------+------------------+-----------
             1 | Academics        |
             2 | DBMS             |         1
             3 | Machine Learning |         1
             4 | Personal         |
             6 | Finance          |         4
             8 | Projects         |
(6 rows)

(Team_7:CS116-CS124)@[local] [personal_archive] >SELECT item_id, collection_id FROM item_collection WHERE item_id = 4;
 item_id | collection_id
---------+---------------
(0 rows)
```